

Project: Moduler & Packager

Overview:

Create a Python-based Multi-Utility Toolkit that integrates multiple standard modules, custom modules, and packages to perform a variety of tasks. This project will demonstrate the effective use of Python's modular structure and provide utility functions for date-time operations, mathematical computations, random data generation, and unique identifier creation.

Objective:

Explore Built-in Modules: Utilize datetime, time, math, random, and uuid modules for practical tasks.

Custom Modules and Packages:

- Create modular Python scripts for specific functionalities.
- Organize them into packages with an `__init__.py` file to demonstrate package usage.

Script Organization: Implement `__name__` and `__main__` to separate reusable components and main execution logic.

Dynamic Exploration: Use the `dir()` function to explore module attributes dynamically.

Functional Requirements:

Datetime and Time Module:

- Display current date and time in a user-friendly format.
- Calculate the difference between two dates or times.
- Format dates into custom formats using `strftime`.
- Implement a simple stopwatch and countdown timer.

Math Module:

- Perform arithmetic and advanced operations like trigonometry, factorials, and logarithms.
- Solve real-world problems like calculating compound interest and area of geometric shapes.

Random Module:

- Generate random numbers, lists, and passwords.

- Simulate random sampling from a dataset.
- Create random OTPs and game simulations.

UUID Module:

- Generate unique identifiers for files, records, or user sessions.
- Demonstrate the use of UUID (e.g., UUID4).

Custom Modules and Packages:

- Build a module for file operations (e.g., reading, writing, and updating files).
- Create another module for mathematical utilities like unit conversions or advanced calculations.
- Group these modules into a package with an `__init__.py` file to initialize the package.

Dynamic Module Exploration:

- Implement a feature where users can explore available attributes of any built-in or custom module using `dir()`.

name and main:

- Separate reusable functions and utilities into different files.
- Use the `__name__` and `__main__` paradigm to execute specific tasks only when the script is run directly.

User Interface (UI):

- Create a menu-driven interface to let users choose from different options.
- Allow users to exit the program from the main menu.

Project Deliverables:

1. Python source code with a menu-driven interface for users to interact with the toolkit.
2. Custom modules and packages demonstrating Python's modular structure.
3. Example outputs of toolkit operations (e.g., logs, reports, or generated identifiers).

Evaluation Criteria:

1. Functionality: Implementation of all required modules and their features.
2. Code Structure: Effective use of modularity, packages, and the `__name__` paradigm.
3. Documentation: Code comments, a README file for the project, and a user guide for the toolkit.
4. Innovation: Creative application of the modules in solving real-world problems.

Example Use Case:

A small business owner uses the toolkit to manage their daily operations. They:

- Calculate working hours using the time module.
- Generate random passwords for their team.
- Assign unique identifiers to invoices using the UUID module.
- Save all logs and outputs into text files via a custom module.
- Dynamically explore the capabilities of the toolkit using dir().

Example Console Interaction:

Menu:

```
=====
Welcome to Multi-Utility Toolkit
=====
Choose an option:
1. Datetime and Time Operations
2. Mathematical Operations
3. Random Data Generation
4. Generate Unique Identifiers (UUID)
5. File Operations (Custom Module)
6. Explore Module Attributes (dir())
7. Exit
=====
Enter your choice: 1
```

Datetime and Time Operations:

```
Datetime and Time Operations:
1. Display current date and time
2. Calculate difference between two dates/times
3. Format date into custom format
4. Stopwatch
5. Countdown Timer
6. Back to Main Menu
Enter your choice: 1

Current Date and Time: 2025-01-04 10:15:30
=====
Enter your choice: 2

Enter the first date (YYYY-MM-DD): 2024-12-25
Enter the second date (YYYY-MM-DD): 2025-01-04
Difference: 10 days
=====
```

Enter your choice: 6

Mathematical Operations:

Mathematical Operations:

1. Calculate Factorial
2. Solve Compound Interest
3. Trigonometric Calculations
4. Area of Geometric Shapes
5. Back to Main Menu

Enter your choice: 1

Enter a number: 5

Factorial: 120

=====

Enter your choice: 2

Enter principal amount: 1000

Enter rate of interest (in %): 5

Enter time (in years): 2

Compound Interest: 1102.50

=====

Enter your choice: 5

Random Data Generation:

Random Data Generation:

1. Generate Random Number
2. Generate Random List
3. Create Random Password
4. Generate Random OTP
5. Back to Main Menu

Enter your choice: 3

Enter password length: 8

Generated Password: Kd8@qzP!

=====

Enter your choice: 5

Generate Unique Identifiers (UUID):

Generate Unique Identifiers:

Generated UUID: f47ac10b-58cc-4372-a567-0e02b2c3d479

=====

Enter your choice: 3

File Operations (Custom Module):

File Operations:

1. Create a new file
2. Write to a file
3. Read from a file
4. Append to a file
5. Back to Main Menu

Enter your choice: 1

Enter file name: example.txt

File created successfully!

=====

Enter your choice: 2

Enter file name: example.txt

Enter data to write: This is a sample file.

Data written successfully!

=====

Enter your choice: 3

Enter file name: example.txt

File Content:

This is a sample file.

=====

Enter your choice: 5

Explore Module Attributes (dir()):

Explore Module Attributes:

Enter module name to explore: math

Available Attributes in math module:

['acos', 'acosh', 'asin', 'asinh', 'atan', 'atanh', 'ceil', 'comb',
'copysign', 'cos', ...]

=====

Exit:

=====
Thank you for using the Multi-Utility Toolkit!
=====

Instructions:

Task Completion: Ensure that you attempt all the assigned tasks given to you as part of the exam. Complete each task to the best of your ability, following the instructions provided.

Assumptions: Make suitable assumptions wherever necessary, based on the requirements and instructions provided. Document any assumptions made in your project documentation or README.md file.

GitHub Repository: Create a GitHub repository to host your project. Upload your project files, including source code, and documentation to the repository. Ensure that you provide a clear and descriptive README.md file.

No Copying: Do not copy code or any other content from your classmates or any other source. Plagiarism is strictly prohibited and can result in severe consequences, including academic penalties. Ensure that all the code and content in your project are original and properly attributed to the appropriate sources, if applicable.

Submission: Once you have completed your project and uploaded it to your GitHub repository, submit the GitHub repository link to your instructor or as instructed. Double-check that your repository is properly organized and includes all the required files as per the instructions provided.

Remember to follow the instructions provided professionally, make suitable assumptions wherever necessary, and avoid copying code or content from unauthorized sources. Good luck with your project work!